

Pipeline

Preprocessing for
Self-supervised learning

At-a-glance



SET-UP

py env
Linux & Windows wrapper
Resource awareness

STEP1

Systems



DOWNLOAD

HAR: streamed
EEG: parallel HTTP
ECG: pooled HTTP

STEP2

Repositories



PRE-PROCESSING

Parsing + wrangling,
harmonization, metadata
generation, dataset adaptor

Event window null
handling+ normalization

Patient-safe split

STEP3

Data prep



VALIDATION

Schema, integrity,
modality specific, config
compliance

STEP4

Output check



OUTPUT ASSETS

Fixed-shape outputs [N,C,T]
(float32) + metadata CSV

Manifests + logs + reports

STEP5

Downstream
use



Pipeline

At-a-glance



SET-UP

Python venv
Linux & Windows
wrapper
Resource awareness

STEP1

systems



- Reproducible & modular
- Python version + dependency conflicts
- Creates folder structure if not there
- Fall back python versions
- Windows powershell + Linux/WSL
- 2 modes:
 - Strict > stop if fail
 - Best-effort > skip if fail and continue the rest

```
project_root/  
|-- setup_data.sh  
|-- setup_data.ps1  
|-- setup_data.py  
|-- preprocess.sh  
|-- preprocess.ps1  
|-- preprocess.py  
|-- validate_outputs.sh  
|-- validate_outputs.ps1  
|-- validate_outputs.py  
|-- configs/  
|-- data/  
|   |-- raw/  
|   |-- interim/  
|   |-- processed/  
|   `-- logs/  
|-- reports/  
|-- scripts/  
|-- submission_sample/  
|-- src/  
|   |-- mmprep/  
|       |-- common/  
|       |-- har/  
|       |-- eeg/  
|       |-- ecg/  
|-- tests/
```

Pipeline

Preprocessing for
Self-supervised learning



At-a-glance

SET-UP

py env
Linux & Windows wrapper
Resource awareness

STEP1

Systems



DOWNLOAD

HAR: streamed
EEG: parallel HTTP
ECG: pooled HTTP

STEP2

Repositories



- Resumable (.part)
- Mode: strict
- Fail cleanly
 - note: file size unknown > corrupt file > retry
- Unzip > nested > recursive clean-up
- Parallelization not too aggressive
- File version from URL > if not there - repo ID
- EEGMMIDB requested subject/run EDF files
- PTB-XL only the selected waveform tree

Pipeline

Preprocessing for
Self-supervised learning

At-a-glance



- HAR:
 - shared 6-channel schema, 20 Hz, harmonized labels, normalized windows, null and class 0, 10s unlabeled pretrain - 5s supervised with 50% overlap
- EEG:
 - 160 Hz, runs 4/8/12, 4 s T1/T2 event windows (can add T0), light preprocessing (average re-reference, notch - 60, band pass (1 → slow drift - 40 → high frequency noise))
- ECG:
 - 100 Hz, native PTB-XL fold metadata (10), patient-safe split, lead normalization, highest weight SCP code

PRE-PROCESSING

Parsing + wrangling,
harmonization, metadata
generation, dataset adaptor

Event window null
handling+ normalization

Patient-safe split

STEP3

Data prep



- HAR > file-by-file (parsing + window), final bundle assembly per output bundle
- EEG > EDF-by-EDF + event-by-event
- ECG > record-by-record
- Interim:
 - HAR → harmonized CSV before windowing
 - EEG → parsed event CSV with metadata + summary JSON (sample count, channel count, sampling rate, channel names, and event count)
 - ECG → summary JSON (per-record summary: record ID, patient ID, number of channels, number of samples, sampling rate, and lead names)

Pipeline

Preprocessing for
Self-supervised learning

At-a-glance



- Tests:
 - Smoke tests: setup-stage directory, manifest, preprocessing (root entrypoints)
 - Unit tests: parser inputs, alternative col names, metadata fields, function outputs
 - Regression tests: unified HAR labels, interim/report outputs, tolerate jitters for gyroscope + accelerometer
- Validation:
 - HAR windowing checks (length, stride)
 - Asset shape + dtype + metadata fields
 - Label + channel schema
 - Null handling + file path and fidelity
 - EEG annotations + runs + subject count
 - ECG CV-fold + leakage checks
 - Submission sample size + resource use
 - Number of manifests

VALIDATION

Schema, integrity,
modality specific, config
compliance

STEP4

Output check



OUTPUT ASSETS

Fixed-shape outputs [N,C,T]
(float32) + metadata CSV

Manifests + logs + reports

STEP5

Downstream
use



Pipeline

Preprocessing for
Self-supervised learning

At-a-glance



- Reproducibility & Adaptability:
 - Env, manifests, logs, config-aware validation, docs, docstrings, modularity
- Resource awareness:
 - Lightweight interim, streaming/chunking, remove zip, storage conscious rate + preprocess choice + context manifest showing machine info for latest run
- Outcome:
 - Reproducible, inspectable, SSL-ready baseline (next: load npz + CSV metadata -> returns (X, metadata) -> pretraining)

OUTPUT ASSETS

Fixed-shape outputs [N,C,T]
(float32) + metadata CSV

Manifests + logs + reports

STEP5

Downstream
use

